

AWS Deployment Runbook

Readable PDF export of the deployment documentation

Yes. Below is a **complete AWS deployment record/runbook** for the system we have built, including the architecture, resources, IDs, configuration, deployment/update procedure, testing, HTTPS/domain migration, Meta/WhatsApp migration, and rollback.

I am keeping **secrets themselves out** of this document. It records the secret names and where they are injected, not their values.

AWS Production Setup — ChatBot Backend

Quick reference

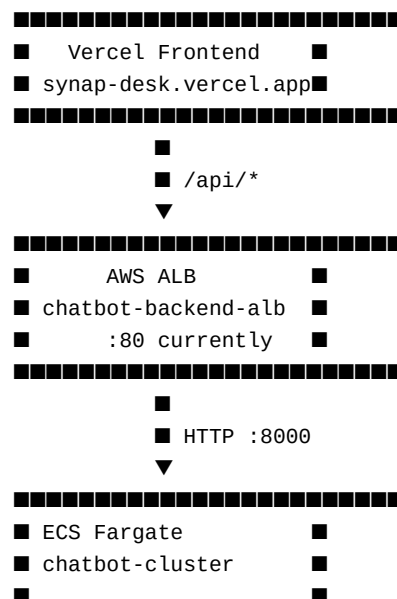
Item	Value
---	---
AWS account	669394142262
AWS region	ap-south-1
AWS CLI profile	chatbot-client
VPC ID	vpc-0496b50752a9b865a
Public subnets	subnet-02e0d652ce39a1acf, subnet-05f22a7f47c7124c2, subnet-05cd38eed67dc6bef
ECR repository	chatbot-backend
ECS cluster	chatbot-cluster
ECS service	chatbot-backend-service
ALB	chatbot-backend-alb
Target group	chatbot-backend-tg
ACM domain	api.svprofessional.in
Secret	chatbot/production
Final WhatsApp callback	https://api.svprofessional.in/api/integrations/sv_professionals/whatsapp

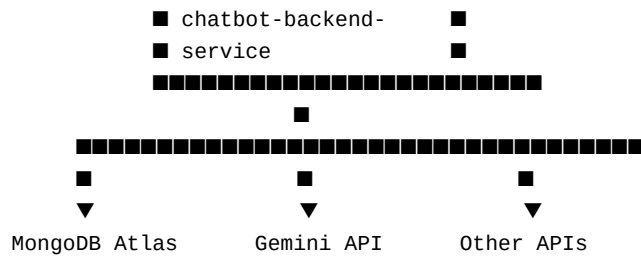
How to use this runbook

- Start with the architecture and infrastructure sections.
- Use the environment and resource IDs for deployment and troubleshooting.
- Follow the ECS/ECR update steps for normal backend releases.
- Only switch Meta to the final HTTPS callback after DNS and ACM are verified.

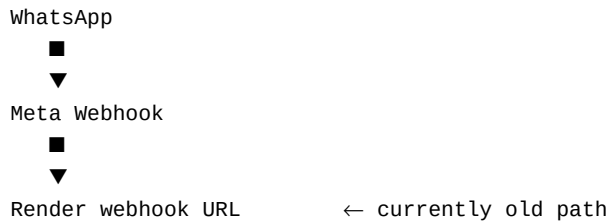
1. Final architecture

Current backend architecture:

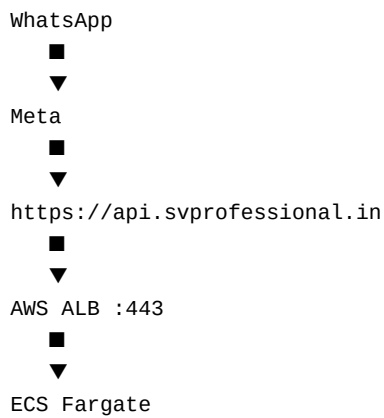




WhatsApp is currently separate:



After HTTPS/domain migration:



2. AWS account / region

AWS account

Account ID:
669394142262

AWS CLI profile

chatbot-client

Region

ap-south-1

Mumbai.

Always use:

--profile chatbot-client --region ap-south-1

because the default AWS CLI profile was not valid.

3. Basic AWS verification

```
aws sts get-caller-identity `
--profile chatbot-client `
--region ap-south-1
```

Expected account:

669394142262

Check region:

```
aws configure get region --profile chatbot-client
```

4. VPC

We are currently using the default VPC.

VPC ID:

vpc-0496b50752a9b865a

CIDR:

172.31.0.0/16

Check:

```
aws ec2 describe-vpcs `
--vpc-ids vpc-0496b50752a9b865a `
--profile chatbot-client `
--region ap-south-1 `
--query "Vpcs[0].{VpcId:VpcId,Cidr:CidrBlock,State:State}" `
--output table
```

5. Public subnets

We use three public subnets.

Mumbai AZ 1a

subnet-02e0d652ce39a1acf
172.31.32.0/20

Mumbai AZ 1b

subnet-05f22a7f47c7124c2
172.31.0.0/20

Mumbai AZ 1c

subnet-05cd38eed67dc6bef
172.31.16.0/20

All currently have:

MapPublicIpOnLaunch = true

Check:

```
aws ec2 describe-subnets `
--subnet-ids `
    subnet-02e0d652ce39a1acf `
    subnet-05f22a7f47c7124c2 `
    subnet-05cd38eed67dc6bef `
--profile chatbot-client `
--region ap-south-1 `
--query "Subnets[].[SubnetId:SubnetId,AZ:AvailabilityZone,CIDR:CidrBlock,PublicIP:MapPublicIpOnLaunch]" `
--output table
```

6. Security groups

ALB security group

sg-0fcff6915f84c0e38

Inbound:

TCP 80

Source: 0.0.0.0/0

Eventually:

TCP 443

Source: 0.0.0.0/0

Check:

```
aws ec2 describe-security-groups `
--group-ids sg-0fcff6915f84c0e38 `
--profile chatbot-client `
--region ap-south-1 `
--output json
```

ECS security group

sg-07cd098e32a96c50e

Inbound:

TCP 8000

Source: sg-0fcff6915f84c0e38

So the Internet cannot directly access ECS port 8000.

Only the ALB can reach it.

Check:

```
aws ec2 describe-security-groups `
--group-ids sg-07cd098e32a96c50e `
--profile chatbot-client `
--region ap-south-1 `
--output json
```

7. ECR

Repository:

chatbot-backend

URI:

669394142262.dkr.ecr.ap-south-1.amazonaws.com/chatbot-backend

ARN:

arn:aws:ecr:ap-south-1:669394142262:repository/chatbot-backend

Check:

```
aws ecr describe-repositories `
  --repository-names chatbot-backend `
  --profile chatbot-client `
  --region ap-south-1 `
  --output table
```

8. ECR configuration

Current important setting:

Tag mutability: IMMUTABLE
Encryption: AES256

Therefore:

> You cannot push a new image using an existing tag until that tag is deleted.

We have been using:

prod-002

Current image digest when originally deployed:

sha256:02f56f8c1be106b273b072aee40441d633cc95e2c480c9c7f8e347f6747cf9c9

Check images:

```
aws ecr describe-images `
  --repository-name chatbot-backend `
  --profile chatbot-client `
  --region ap-south-1 `
  --query "imageDetails[].[Tags:imageTags,Size:imageSizeInBytes,Pushed:imagePushedAt,Digest:imageDigest]" `
  --output table
```

9. ECR login

Set variables:

```
$REGION="ap-south-1"
$PROFILE="chatbot-client"
$ACCOUNT_ID="669394142262"
$REPO="chatbot-backend"
$IMAGE_URI="$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/$REPO"
```

Login:

```
aws ecr get-login-password `
  --profile $PROFILE `
  --region $REGION |
docker login `
  --username AWS `
  --password-stdin $IMAGE_URI
```

Expected:

Login Succeeded

10. Docker image

Backend Dockerfile is under:

backend/Dockerfile

Build context:

backend/

The container:

```
Python 3.11
FastAPI
Gunicorn
Uvicorn worker
PORT=8000
```

Application binds:

0.0.0.0:8000

Docker health check:

/healthz

11. Build image

From:

C:\ChatX\backend

for example:

```
$TAG="prod-002"
```

```
docker build -t "$IMAGE_URI`:$TAG" .
```

Test locally:

```
docker run --rm -p 8000:8000 "$IMAGE_URI`:$TAG"
```

Then in another PowerShell window:

```
Invoke-RestMethod `
  -Uri "http://localhost:8000/healthz"
```

Expected:

```
{
  "status": "ok",
  "service": "rag-backend"
}
```

Stop the local container with:

Ctrl+C

12. Push image

```
docker push "$IMAGE_URI`:$TAG"
```

Get digest:

```
aws ecr describe-images `
  --repository-name chatbot-backend `
  --image-ids imageTag=$TAG `
  --profile $PROFILE `
  --region $REGION `
  --query "imageDetails[0].imageDigest" `
  --output text
```

13. Reusing `prod-002`

Because ECR is immutable:

```
aws ecr batch-delete-image `
  --repository-name chatbot-backend `
  --image-ids imageTag=prod-002 `
  --profile chatbot-client `
  --region ap-south-1
```

Then:

```
docker build -t "$IMAGE_URI`:prod-002" .
```

and:

```
docker push "$IMAGE_URI`:prod-002"
```

Important

The **ECS task definition should still get a new revision**.

For example:

ECS task definition

```
chatbot-backend:1
chatbot-backend:2
chatbot-backend:3
```

This is intentional and gives us rollback capability.

The ECR image tag can be reused, but ECS revision history should not be deleted/reused.

14. ECS cluster

Cluster:

chatbot-cluster

ARN:

arn:aws:ecs:ap-south-1:669394142262:cluster/chatbot-cluster

Check:

```
aws ecs describe-clusters `
  --clusters chatbot-cluster `
  --profile chatbot-client `
  --region ap-south-1 `
  --output table
```

15. ECS service

Service:

chatbot-backend-service

Cluster:

chatbot-cluster

ARN:

arn:aws:ecs:ap-south-1:669394142262:service/chatbot-cluster/chatbot-backend-service

Check:

```
aws ecs describe-services `
  --cluster chatbot-cluster `
  --services chatbot-backend-service `
  --profile chatbot-client `
  --region ap-south-1 `
  --query "services[0].{Status:status,Desired:desiredCount,Running:runningCount,Pending:pendingCount,TaskDefinition:taskDefinitionArn}" `
  --output table
```

Expected:

Status	Desired	Running	Pending
ACTIVE	1	1	0

16. ECS compute

Current configuration:

Launch type: FARGATE
CPU: 1024
Memory: 2048 MB
Desired tasks: 1

Equivalent:

1 vCPU

2 GB RAM

Deployment:

Deployment circuit breaker: enabled
Rollback: enabled
Minimum healthy: 100%
Maximum: 200%

This means during deployment ECS can temporarily run old + new task.

17. ECS networking

Task uses:

awsvpc

Subnets:

subnet-02e0d652ce39a1acf
subnet-05f22a7f47c7124c2
subnet-05cd38eed67dc6bef

Security group:

sg-07cd098e32a96c50e

Public IP:

enabled

This is our initial simple production architecture.

Later, for hardening, the ECS task can be moved to private subnets with NAT Gateway/VPC endpoints.

18. ECS task definition

Family:

chatbot-backend

Current initial revision:

chatbot-backend:1

Check current revision:

```
aws ecs describe-task-definition `
  --task-definition chatbot-backend `
  --profile chatbot-client `
  --region ap-south-1 `
  --query "taskDefinition.{Family:family,Revision:revision,CPU:cpu,Memory:memory,Network:networkMode,Image:containerImage}"
  --output table
```

19. Container

Container name:

chatbot-backend

Port:

8000

Health check:

```
curl -f http://localhost:8000/healthz || exit 1
```

Configuration:

```
interval: 30 sec
timeout: 5 sec
retries: 3
startPeriod: 60 sec
```

20. IAM execution role

Role:

ecsTaskExecutionRole

ARN:

arn:aws:iam::669394142262:role/ecsTaskExecutionRole

Trust:

ecs-tasks.amazonaws.com

Managed policy:

AmazonECSTaskExecutionRolePolicy

This gives ECS the ability to:

- * pull ECR image
- * write CloudWatch logs
- * retrieve required AWS resources for task startup

21. Secrets Manager

Secret:

chatbot/production

ARN:

arn:aws:secretsmanager:ap-south-1:669394142262:secret:chatbot/production-04c9pA

It contains:

```
MONGODB_URI
MONGODB_DB_NAME
GEMINI_API_KEY
GROQ_API_KEY
JWT_SECRET
PLATFORM_SETUP_KEY
```

No secrets are hard-coded in the ECS task definition.

Check metadata only:

```
aws secretsmanager describe-secret `
  --secret-id chatbot/production `
  --profile chatbot-client `
  --region ap-south-1 `
  --query "{Name:Name,ARN:ARN,LastChanged:LastChangedDate}" `
  --output table
```

****Do not use `get-secret-value` and paste its output into chat.****

22. IAM secret access

Inline policy:

ChatbotProductionSecretsAccess

It permits:

secretsmanager:GetSecretValue

on:

chatbot/production

This is attached to:

ecsTaskExecutionRole

23. ECS environment configuration

Non-secret configuration currently includes:

LLM_PROVIDER=gemini

EMBEDDING_PROVIDER=huggingface

VECTORDB_PROVIDER=mongodb

GEMINI_MODEL=models.gemini-pro-latest

GROQ_MODEL=groq/compound

HUGGINGFACE_MODEL=sentence-transformers/all-MiniLM-L6-v2

MONGODB_DB_NAME=ChatBot

MONGODB_VECTOR_INDEX_NAME=default_vector_index

HYDE_ENABLED=true

RERANK_ENABLED=true

CACHE_ENABLED=true

CACHE_SIMILARITY_THRESHOLD=0.85

CACHE_TTL_HOURS=24

RETRIEVAL_CANDIDATES=30

RETRIEVAL_TOP_K=8

```
ALLOWED_ORIGINS=  
http://localhost:3000,  
http://127.0.0.1:5500,  
https://synap-desk.vercel.app
```

24. CloudWatch

Log group:

/ecs/chatbot-backend

Check:

```
aws logs describe-log-groups `--log-group-name-prefix "/ecs/chatbot-backend" `--profile chatbot-client `--region ap-south-1 `--output table
```

Tail logs:

```
aws logs tail /ecs/chatbot-backend `--profile chatbot-client `--region ap-south-1 `--since 15m
```

Live:

```
aws logs tail /ecs/chatbot-backend `--profile chatbot-client `--region ap-south-1 `--since 2m `--follow
```

25. Application startup

The ECS logs previously showed:

```
unicorn  
UvicornWorker  
0.0.0.0:8000  
Application startup complete
```

This confirms the FastAPI application starts correctly inside ECS.

26. Application health endpoint

Direct ALB:

```
Invoke-RestMethod `--Uri "http://chatbot-backend-alb-241464516.ap-south-1.elb.amazonaws.com/healthz"
```

Expected:

```
{  
  "status": "ok",  
  "service": "rag-backend"
```

```
}
```

27. Application provider health

```
Invoke-RestMethod `
  -Uri "http://chatbot-backend-alb-241464516.ap-south-1.elb.amazonaws.com/api/health"
```

Expected:

```
{
  "status": "healthy",
  "providers": {
    "llm": "gemini",
    "embedding": "huggingface",
    "vectordb": "mongodb"
  },
  "version": "0.1.0"
}
```

This test proved:

```
ALB
↓
ECS
↓
FastAPI
↓
MongoDB
```

is working.

28. Authentication test

Endpoint:

POST /api/auth/login

Test directly against AWS:

```
$body = @{
  email    = "YOUR_EMAIL"
  password = "YOUR_PASSWORD"
} | ConvertTo-Json
```

```
$response = Invoke-RestMethod `
  -Uri "http://chatbot-backend-alb-241464516.ap-south-1.elb.amazonaws.com/api/auth/login" `
  -Method POST `
  -ContentType "application/json" `
  -Body $body
```

\$response

Do not paste the resulting JWT into chat.

29. Setup-status test

```
Invoke-RestMethod `
```

```
-Uri "http://chatbot-backend-alb-241464516.ap-south-1.elb.amazonaws.com/api/auth/setup-status"
```

Previously returned:

```
{  
  "setup_required": false  
}
```

30. ALB

ALB:

chatbot-backend-alb

ARN:

arn:aws:elasticloadbalancing:ap-south-1:669394142262:loadbalancer/app/chatbot-backend-alb/d0c54ada171b1b0b

DNS:

chatbot-backend-alb-241464516.ap-south-1.elb.amazonaws.com

Type:

Application Load Balancer
Internet-facing
IPv4

31. ALB target group

Target group:

chatbot-backend-tg

ARN:

arn:aws:elasticloadbalancing:ap-south-1:669394142262:targetgroup/chatbot-backend-tg/6552c12865dc429c

Configuration:

Protocol: HTTP
Port: 8000
Target type: IP
Health path: /healthz
Success code: 200

Check:

```
aws elbv2 describe-target-health \  
  --target-group-arn "arn:aws:elasticloadbalancing:ap-south-1:669394142262:targetgroup/chatbot-backend-tg/6552c12865dc429c" \  
  --profile chatbot-client \  
  --region ap-south-1 \  
  --output table
```

Currently the target was:

172.31.42.227:8000
healthy

The IP can change when ECS replaces the task, so don't hard-code it.

32. ALB listener

Current listener:

HTTP :80

Listener ARN:

arn:aws:elasticloadbalancing:ap-south-1:669394142262:listener/app/chatbot-backend-alb/d0c54ada171b1b0b/0a2305896ee

It forwards to:

chatbot-backend-tg

33. ALB access logging

Currently:

```
access_logs.s3.enabled = false
```

Check:

```
aws elbv2 describe-load-balancer-attributes `
  --load-balancer-arn "arn:aws:elasticloadbalancing:ap-south-1:669394142262:loadbalancer/app/chatbot-backend-alb/c
  --profile chatbot-client `
  --region ap-south-1 `
  --query "Attributes[?Key=='access_logs.s3.enabled']" `
  --output table
```

We have **not enabled ALB S3 access logs**.

34. Current production Vercel relationship

Vercel frontend:

<https://synap-desk.vercel.app>

Environment variable:

NEXT_PUBLIC_API_URL

The browser itself showed:

POST <https://synap-desk.vercel.app/api/auth/login>

So the frontend uses a same-origin `/api` route rather than directly displaying the AWS ALB URL in the browser.

The important practical test we performed:

Render suspended

↓

Vercel login still works

↓

Other application functions still work

This is strong evidence that the application's backend dependency has moved away from Render, although the

exact Vercel proxy implementation should be documented in the frontend repository.

35. Render

Render is currently being kept as rollback/legacy infrastructure.

Current strategy:

Render



KEEP AVAILABLE UNTIL FULL AWS CUTOVER

For WhatsApp, Meta is/was configured with the old Render callback:

https://synapdesk.onrender.com/api/integrations/sv_professionals/whatsapp

Therefore WhatsApp migration is a separate final step.

36. Current WhatsApp API route

Backend route:

/api/integrations/sv_professionals/whatsapp

Current old Meta callback:

https://synapdesk.onrender.com/api/integrations/sv_professionals/whatsapp

Final intended callback:

https://api.svprofessional.in/api/integrations/sv_professionals/whatsapp

But ****do not change Meta to the final URL until HTTPS is working****.

37. ACM certificate

Certificate requested in:

ap-south-1

ARN:

arn:aws:acm:ap-south-1:669394142262:certificate/99b9c757-5b1e-4829-8d32-c7768ba4d50b

Domain:

api.svprofessional.in

38. ACM DNS validation record

The required validation record is:

Type:

CNAME

Name:


```
_cfd5e86ad4c0a311bf06fa7eb1030c5e.api.svprofessional.in
```

Value:

```
_f855514d39031db455a5fa1b811cd4d0.wzccmgtwzk.acm-validations.aws.
```

The client needs to add this in the ****authoritative DNS provider****.

You discovered that the domain may be registered at GoDaddy, but the authoritative nameservers previously returned were:

```
ns1.cp-ht-8.hostgatorwebservers.com
```

```
ns2.cp-ht-8.hostgatorwebservers.com
```

Therefore verify before adding anything:

```
nslookup -type=NS svprofessional.in
```

If those HostGator nameservers are still returned, the DNS record must be added where those nameservers are managed.

39. Check ACM status

```
aws acm describe-certificate `
  --certificate-arn "arn:aws:acm:ap-south-1:669394142262:certificate/99b9c757-5b1e-4829-8d32-c7768ba4d50b" `
  --profile chatbot-client `
  --region ap-south-1 `
  --query "Certificate.{Domain:DomainName,Status:Status,Validation:DomainValidationOptions[0].ResourceRecord}" `
  --output json
```

We want:

```
ISSUED
```

40. HTTPS ALB setup

Once ACM says:

```
ISSUED
```

create an HTTPS listener:

```
ALB :443
  ↓
ACM certificate
  ↓
chatbot-backend-tg :8000
```

Before doing this, ALB SG needs:

```
TCP 443
0.0.0.0/0
```

Then create the listener:

```
aws elbv2 create-listener `
  --load-balancer-arn "arn:aws:elasticloadbalancing:ap-south-1:669394142262:loadbalancer/app/chatbot-backend-alb/c"
  --protocol HTTPS `
  --port 443 `
```

```
--certificates CertificateArn="arn:aws:acm:ap-south-1:669394142262:certificate/99b9c757-5b1e-4829-8d32-c7768ba40
--default-actions Type=forward,TargetGroupArn="arn:aws:elasticloadbalancing:ap-south-1:669394142262:targetgroup/
--profile chatbot-client `
--region ap-south-1
```

Then eventually HTTP :80 should redirect to HTTPS instead of forwarding directly.

41. Final DNS

Final desired record:

```
api.svprofessional.in
      ↓
AWS ALB
```

Depending on the DNS provider, this is normally configured as an appropriate CNAME/ALIAS record.

Do not blindly create an A record containing the ALB hostname.

After DNS is configured:

```
nslookup api.svprofessional.in
```

Then:

```
Invoke-RestMethod `
  -Uri "https://api.svprofessional.in/healthz"
```

Expected:

```
{
  "status": "ok",
  "service": "rag-backend"
}
```

42. Final Vercel configuration

Once HTTPS works:

```
NEXT_PUBLIC_API_URL
```

should ultimately use:

```
https://api.svprofessional.in
```

Then redeploy Vercel because `NEXT\_PUBLIC\_\*` variables are build-time frontend configuration.

After deployment, browser Network should show either:

```
https://api.svprofessional.in/api/...
```

if direct API access is used, or the Vercel same-origin proxy should be confirmed to forward to that AWS endpoint.

43. Final Meta configuration

After HTTPS is verified:

Meta WhatsApp callback:

`https://api.svprofessional.in/api/integrations/sv_professionals/whatsapp`

Meta verification:

Callback URL:

`https://api.svprofessional.in/api/integrations/sv_professionals/whatsapp`

Verify Token:

<the application's configured WhatsApp verification token>

Do not paste the verify token into chat.

Meta will perform the GET verification handshake.

Then incoming messages:

```
whatsApp
  ↓
Meta
  ↓
AWS ALB :443
  ↓
ECS
  ↓
FastAPI WhatsApp integration
  ↓
MongoDB / Gemini
  ↓
Meta
  ↓
WhatsApp response
```

44. Updating backend code — normal procedure

For every new backend change:

```
Modify code
  ↓
Local test
  ↓
Docker build
  ↓
ECR push
  ↓
New ECS task-definition revision
  ↓
Update ECS service
  ↓
ECS rolling deployment
  ↓
Health check
  ↓
Functional test
```

Set variables

```
$REGION="ap-south-1"
$PROFILE="chatbot-client"
$ACCOUNT_ID="669394142262"
$REPO="chatbot-backend"
$IMAGE_URI="$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/$REPO"
```

```
$TAG="prod-002"
```

Delete old immutable tag

```
aws ecr batch-delete-image `
  --repository-name $REPO `
  --image-ids imageTag=$TAG `
  --profile $PROFILE `
  --region $REGION
```

Build

```
docker build -t "$IMAGE_URI`:$TAG" .
```

Push

```
docker push "$IMAGE_URI`:$TAG"
```

Get digest

```
$NEW_DIGEST = aws ecr describe-images `
  --repository-name $REPO `
  --image-ids imageTag=$TAG `
  --profile $PROFILE `
  --region $REGION `
  --query "imageDetails[0].imageDigest" `
  --output text
```

```
$NEW_DIGEST
```

45. Create new ECS revision after code update

Export current task definition:

```
aws ecs describe-task-definition `
  --task-definition chatbot-backend `
  --profile $PROFILE `
  --region $REGION `
  --query "taskDefinition" `
  --output json > task-definition-current.json
```

Modify its container image to:

```
669394142262.dkr.ecr.ap-south-1.amazonaws.com/chatbot-backend@sha256:NEW_DIGEST
```

Then register the new revision.

After registration:

```
$REGISTERED = aws ecs register-task-definition `
  --cli-input-json file://task-definition-new.json `
  --profile $PROFILE `
  --region $REGION `
  --query "taskDefinition.taskDefinitionArn" `
  --output text
```

```
$REGISTERED
```

46. Deploy new ECS revision

```
aws ecs update-service `
  --cluster chatbot-cluster `
  --service chatbot-backend-service `
  --task-definition $REGISTERED `
  --profile $PROFILE `
  --region $REGION
```

Wait:

```
aws ecs wait services-stable `
  --cluster chatbot-cluster `
  --services chatbot-backend-service `
  --profile $PROFILE `
  --region $REGION
```

Then:

```
aws ecs describe-services `
  --cluster chatbot-cluster `
  --services chatbot-backend-service `
  --profile $PROFILE `
  --region $REGION `
  --query "services[0].{Status:status,Desired:desiredCount,Running:runningCount,Pending:pendingCount,TaskDefinition:taskDefinitionArn}" `
  --output table
```

47. Verify the deployed image

```
aws ecs describe-task-definition `
  --task-definition $REGISTERED `
  --profile $PROFILE `
  --region $REGION `
  --query "taskDefinition.containerDefinitions[0].image" `
  --output text
```

It should contain:

```
@sha256:<new digest>
```

48. Verify ECS task

Get task:

```
aws ecs list-tasks `
  --cluster chatbot-cluster `
  --service-name chatbot-backend-service `
  --profile chatbot-client `
  --region ap-south-1 `
  --output table
```

Then:

```
aws ecs describe-tasks `
  --cluster chatbot-cluster `
  --tasks <TASK_ARN> `
  --profile chatbot-client `
  --region ap-south-1 `
```

```
--query "tasks[0].{Status:lastStatus,Health:healthStatus,TaskDefinition:taskDefinitionArn}" `
--output table
```

49. Test after every deployment

ALB health

```
Invoke-RestMethod `
  -Uri "http://chatbot-backend-alb-241464516.ap-south-1.elb.amazonaws.com/healthz"
```

Application health

```
Invoke-RestMethod `
  -Uri "http://chatbot-backend-alb-241464516.ap-south-1.elb.amazonaws.com/api/health"
```

Logs

```
aws logs tail /ecs/chatbot-backend `
  --profile chatbot-client `
  --region ap-south-1 `
  --since 5m
```

50. Rollback

This is one reason we keep ECS revisions.

List revisions:

```
aws ecs list-task-definitions `
  --family-prefix chatbot-backend `
  --profile chatbot-client `
  --region ap-south-1 `
  --sort DESC `
  --output table
```

Suppose the previous working revision is:

chatbot-backend:2

Rollback:

```
aws ecs update-service `
  --cluster chatbot-cluster `
  --service chatbot-backend-service `
  --task-definition chatbot-backend:2 `
  --profile chatbot-client `
  --region ap-south-1
```

Wait:

```
aws ecs wait services-stable `
  --cluster chatbot-cluster `
  --services chatbot-backend-service `
  --profile chatbot-client `
  --region ap-south-1
```

This is much safer than trying to rebuild an old image.

51. Current application routes

The deployed OpenAPI exposes major areas including:

/healthz

/api/health

/api/documents/*

/api/chat/*

/api/auth/*

/api/clients/*

/api/analytics/*

/api/integrations/*

Authentication:

POST /api/auth/login

Chat:

POST /api/chat/{client_id}/query

WhatsApp:

/api/integrations/sv_professionals/whatsapp

52. Important current application issue

We discovered this when testing AWS chat directly:

Institution 'default' not found

This was ****not an AWS infrastructure failure****.

The AWS application successfully:

started

→ connected to MongoDB

→ authenticated

→ reached chat logic

→ attempted to find institution

but `default` was not an actual institution/client in the database.

So when testing chat, use the actual client/institution ID rather than:

default

53. Important code cleanup before final production

There are a few things we discovered that should eventually be cleaned up.

CORS

`config.py` has:

ALLOWED_ORIGINS

but `main.py` currently contains a separate hard-coded CORS list and broad HTTPS regex.

Production should ideally have one source of truth:

`settings.cors_origins`

and explicitly allow the real frontend origins.

Stale Render references

The deployed OpenAPI still contains descriptions referring to:

Render
`https://synapdesk.onrender.com`

These do not currently break routing, but they should be removed/updated before final migration.

Production configuration defaults

Some application defaults such as:

`JWT_SECRET="change-me-in-production"`

are safe only when overridden by Secrets Manager.

The production ECS environment already injects the actual secret, but startup validation could be added later to prevent accidental deployment with unsafe defaults.

54. Current security model

Current:

```
Internet
  ↓
ALB :80
  ↓
ECS :8000
```

ECS port 8000 is ****not open to the entire Internet****.

It accepts traffic only from:

`sg-0fcff6915f84c0e38`

The ALB is Internet-facing.

Secrets are in:

AWS Secrets Manager

rather than GitHub/Dockerfile/task-definition plaintext.

55. Current production status

Working

AWS account	■
VPC	■
Subnets	■
Security groups	■
ECR	■

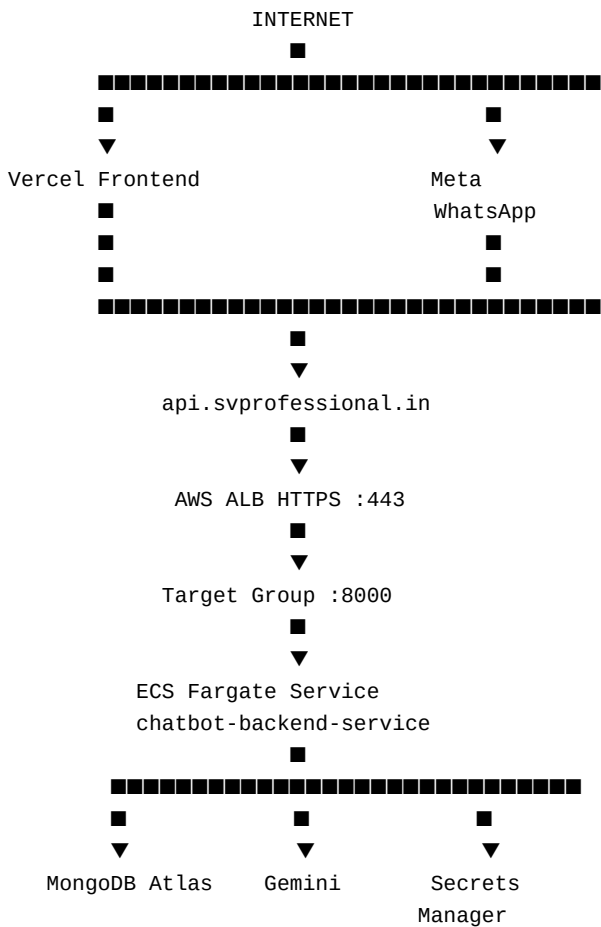
- Docker image
- ECS cluster
- ECS Fargate service
- Task
- ALB
- Target group
- Health check
- MongoDB connection
- Gemini configuration
- Authentication
- CloudWatch logs
- Vercel application path
- strong evidence

Not final yet

- HTTPS :443
- ACM DNS validation
- api.svprofessional.in
- Meta → AWS webhook
- WhatsApp → AWS
- HTTP → HTTPS redirect
- Final CORS cleanup
- Render retirement

56. The final production architecture

Once everything is complete, this is what we want:



Render then becomes:

The operational rule I recommend

For future backend changes, your standard procedure should be:

1. Change code
2. Test locally
3. Build Docker
4. Delete old prod-002 tag
5. Push new prod-002
6. Get new SHA256 digest
7. Create NEW ECS task-definition revision
8. Update ECS service
9. Wait for service stable
10. Verify target healthy
11. Test /healthz
12. Test /api/health
13. Test authentication
14. Test the changed feature
15. Check CloudWatch

****Never delete the previous ECS task-definition revision.**** That revision history is your rollback mechanism.

And for the final migration:

ACM
↓
HTTPS :443
↓
api.svprofessional.in
↓
Vercel → AWS
↓
Meta → AWS
↓
WhatsApp test
↓
Keep Render as rollback
↓
Only then retire Render

That is the complete setup we have established in the client's AWS account so far.

AWS Production Backend Update Procedure

1. Environment

AWS Region

ap-south-1

AWS CLI Profile

chatbot-client

ECR Repository

chatbot-backend

ECS Cluster

chatbot-cluster

ECS Service

chatbot-backend-service

ECS Task Definition Family

chatbot-backend

2. Navigate to Backend

cd C:\ChatX\backend

Get-Location

Expected:

C:\ChatX\backend

3. Set Deployment Variables

\$REGION="ap-south-1"

\$PROFILE="chatbot-client"

\$ACCOUNT_ID="669394142262"

\$REPO="chatbot-backend"

\$IMAGE_URI="\$ACCOUNT_ID.dkr.ecr.\$REGION.amazonaws.com/\$REPO"

\$TAG="prod-002"

\$CLUSTER="chatbot-cluster"

\$SERVICE="chatbot-backend-service"

Verify:

Write-Host "IMAGE_URI = \$IMAGE_URI"

Write-Host "TAG = \$TAG"

Write-Host "CLUSTER = \$CLUSTER"

Write-Host "SERVICE = \$SERVICE"

4. Authenticate Docker With ECR

aws ecr get-login-password --region \$REGION --profile \$PROFILE | docker login --username AWS --password-stdin \$IMAGE_URI

Expected:

Login Succeeded

5. Delete Existing Immutable ECR Tag

Because the repository uses immutable tags, the existing prod-002 tag had to be removed before rebuilding and pushing a new image with the same tag.

aws ecr batch-delete-image --repository-name \$REPO --image-ids imageTag=\$TAG --region \$REGION --profile \$PROFILE

Verify the deletion response contains:

imageTag: prod-002

failures: []

6. Build the New Docker Image

From:

C:\ChatX\backend

run:

docker build -t "\$IMAGE_URI:\$TAG" .

This builds:

669394142262.dkr.ecr.ap-south-1.amazonaws.com/chatbot-backend:prod-002

Verify:

docker images "\$IMAGE_URI:\$TAG"

7. Push the New Image to ECR

docker push "\$IMAGE_URI:\$TAG"

The successful deployment produced:

prod-002

with digest:

```
sha256:701762e3e354e3e4e9db34bc698dcadd1990182a318871340f3ab24b2fde19b5
```

8. Verify the ECR Image Digest

```
aws ecr describe-images `
--repository-name $REPO `
--image-ids imageTag=$TAG `
--region $REGION `
--profile $PROFILE `
--query "imageDetails[0].imageDigest" `
--output text
```

Expected:

```
sha256:701762e3e354e3e4e9db34bc698dcadd1990182a318871340f3ab24b2fde19b5
```

9. Export Current ECS Task Definition

Before changing anything, export the currently running task definition:

```
aws ecs describe-task-definition `
--task-definition chatbot-backend `
--region $REGION `
--profile $PROFILE `
--query "taskDefinition" |
Out-File -Encoding utf8 task-definition-current.json
```

This captured revision 1.

The previous image was:

```
sha256:02f56f8c1be106b273b072aee40441d633cc95e2c480c9c7f8e347f6747cf9c9
```

10. Replace Only the Docker Image Digest

The existing task definition was modified so that the application configuration, environment variables, secrets, ports, health check, logging configuration, etc. remained unchanged.

```
(Get-Content .\task-definition-current.json -Raw) `
-replace '669394142262\.dkr\.ecr\.ap-south-1\.amazonaws\.com/chatbot-backend@sha256:[a-f0-9]+' , `
```

```
'669394142262.dkr.ecr.ap-south-1.amazonaws.com/chatbot-backend@sha256:701762e3e354e3e4e9db34bc698dcadd1990182a318871340f3ab24b2fde19b5'
```

```
|
Set-Content -Encoding utf8 .\task-definition-new.json
```

Verify:

```
Get-Content .\task-definition-new.json | Select-String "image"
```

11. Validate Task Definition JSON

```
Get-Content .\task-definition-new.json -Raw |
ConvertFrom-Json |
Out-Null
```

```
if ($?) {
Write-Host "JSON VALID"
}
```

Expected:

```
JSON VALID
```

12. Remove ECS-Generated Fields

Before registering the new revision:

```
$j = Get-Content .\task-definition-new.json -Raw | ConvertFrom-Json
```

```
$j.PSObject.Properties.Remove("taskDefinitionArn")
$j.PSObject.Properties.Remove("revision")
$j.PSObject.Properties.Remove("status")
$j.PSObject.Properties.Remove("requiresAttributes")
```

```
$j.PSObject.Properties.Remove("compatibilities")
$j.PSObject.Properties.Remove("registeredAt")
$j.PSObject.Properties.Remove("registeredBy")
```

```
$j |
ConvertTo-Json -Depth 20 |
Set-Content -Encoding utf8 .\task-definition-register.json
13. Create Clean UTF-8 JSON
```

Because the AWS CLI on Windows rejected the original PowerShell-generated file encoding, we converted it to clean UTF-8 without BOM:

```
[System.IO.File]::WriteAllText(
(Join-Path (Get-Location) "task-definition-register-clean.json"),
($j | ConvertTo-Json -Depth 20 -Compress),
(New-Object System.Text.UTF8Encoding($false))
)
```

Validate:

```
Get-Content .\task-definition-register-clean.json -Raw |
ConvertFrom-Json |
Out-Null
```

```
if ($?) {
Write-Host "CLEAN JSON VALID"
}
```

14. Register New ECS Task Definition

```
aws ecs register-task-definition `
--cli-input-json file://task-definition-register-clean.json `
--region $REGION `
--profile $PROFILE
```

This created:

chatbot-backend:2

The important point is:

Revision 1 → old production image
Revision 2 → new production image

Revision 1 remains available for rollback.

15. Verify Revision 2

```
aws ecs describe-task-definition `
--task-definition chatbot-backend:2 `
--region $REGION `
--profile $PROFILE `
--query "taskDefinition.{Revision:revision,Image:containerDefinitions[0].image}" `
--output table
```

Confirmed:

Revision = 2
Image = ...@sha256:701762e3...

16. Deploy Revision 2 to ECS

```
aws ecs update-service `
--cluster $CLUSTER `
--service $SERVICE `
--task-definition chatbot-backend:2 `
--region $REGION `
--profile $PROFILE
```

ECS then performed a rolling deployment.

The old task was gradually deregistered while the new task became healthy.

17. Wait for ECS Stability

```
aws ecs wait services-stable `
--cluster $CLUSTER `
--services $SERVICE `
--region $REGION `
--profile $PROFILE
```

No output + return to the PowerShell prompt = successful completion.

18. Verify ECS Deployment

```
aws ecs describe-services `
--cluster $CLUSTER `
--services $SERVICE `
--region $REGION `
--profile $PROFILE `
--query
"services[0].deployments[0].{Status:status,TaskDefinition:taskDefinition,Desired:desiredCount,Running:runningCount,Pending:pendingCount}"
--output table
```

Confirmed:

```
Status PRIMARY
TaskDefinition chatbot-backend:2
Desired 1
Running 1
Pending 0
```

19. Verify Actual Running Container

```
aws ecs describe-tasks `
--cluster $CLUSTER `
--tasks (
aws ecs list-tasks `
--cluster $CLUSTER `
--service-name $SERVICE `
--region $REGION `
--profile $PROFILE `
--query "taskArns[0]" `
--output text
) `
--region $REGION `
--profile $PROFILE `
--query
"tasks[0].{TaskDefinition:taskDefinitionArn,Image:containers[0].image,LastStatus:lastStatus,Health:healthStatus}"
--output table
```

Confirmed:

```
TaskDefinition = chatbot-backend:2
Image = ...@sha256:701762e3...
LastStatus = RUNNING
Health = HEALTHY
```

This is important because it verifies that ECS isn't merely configured for revision 2 — the new container is actually running.

20. Verify ALB Target Health

```
aws elbv2 describe-target-health `
--target-group-arn
arn:aws:elasticloadbalancing:ap-south-1:669394142262:targetgroup/chatbot-backend-tg/6552c12865dc429c `
--region $REGION `
--profile $PROFILE `
--query
"TargetHealthDescriptions[0].{Target:Target.Id,Port:Target.Port,State:TargetHealth.State,Reason:TargetHealth.Reason}"
--output table
```

--output table

Current result:

172.31.30.77:8000 healthy
172.31.42.227:8000 draining

The draining target is the old task being removed from the target group.

The important target is:

172.31.30.77
State: healthy

So this is expected during/just after a rolling deployment.

21. Verify Public ALB Endpoint

curl.exe http://chatbot-backend-alb-241464516.ap-south-1.elb.amazonaws.com/healthz

Confirmed response:

```
{
  "status": "ok",
  "service": "rag-backend"
}
```

Therefore the request path is working:

Internet

↓

ALB

↓

Target Group

↓

New ECS Task

↓

FastAPI

↓

/healthz

Final Deployment State

AWS

■

■■■■ ECR

■ ■■■■ chatbot-backend

■ ■■■■ prod-002

■ ■■■■ sha256:701762e3...

■

■■■■ ECS Cluster

■ ■■■■ chatbot-cluster

■ ■

■ ■■■■ chatbot-backend-service

■ ■

■ ■■■■ chatbot-backend:2

■ ■■■■ RUNNING / HEALTHY

■

■■■■ ALB

■ ■■■■ chatbot-backend-alb

■ ■

■ ■■■■ Target Group

■ ■■■■ 172.31.30.77:8000

■ ■■■■ HEALTHY

■

■■■■ Public health check

■■■■ /healthz

■■■■ {"status":"ok","service":"rag-backend"}

Deployment conclusion

The backend update from the previous image to the rebuilt prod-002 image is complete and verified end-to-end. The old ECS task is draining normally, while the new revision is healthy and serving traffic.

For future deployments, the important operational pattern is:

Build → Push ECR → Get digest → Create new ECS task-definition revision → Update service → Wait stable → Verify running task → Verify ALB health.